

STM32 标准库函数 手写代码速学手册

STM32F10x Standard Peripheral Library
Handwriting Quick Reference

期末考试 · 面试手写代码速查

定位

看到题目 → 回忆配置顺序 → 直接写代码

不写原理 不推导 只记模板

覆盖模块：GPIO · EXTI · TIM · ADC · USART

页数：约 20 页 适合考前打印 A4 复习

版本 2026.06

注意

本手册仅覆盖期末考试 & 面试常见题型
不包含输入捕获、DMA、SPI、I2C、CAN 等内容
所有代码基于 STM32F10x 标准库 3.5.0

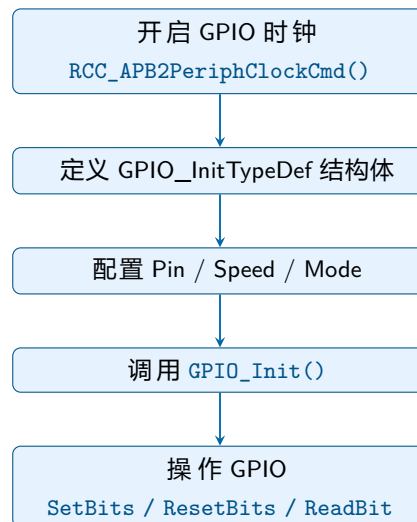
目录

第一章 GPIO（通用输入输出）

一、考试考什么

- LED 亮灭控制（推挽输出）
- 按键检测（浮空输入 / 上拉输入）
- GPIO 初始化结构体填空
- 模式选择（GPIO_Mode）
- 位操作函数选择

二、GPIO 配置流程图



三、万能代码框架

GPIO 初始化模板

```
void GPIO_Config(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOx, ENABLE); // 开时钟

    GPIO_InitStructure.GPIO_Pin   = GPIO_Pin_x;           // 引脚号
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;      // 输出速度
    GPIO_InitStructure.GPIO_Mode  = GPIO_Mode_Out_PP;      // 模式
    GPIO_Init(GPIOx, &GPIO_InitStructure);                // 写入配置
}
```

LED 闪烁模板

```
void LED_Init(void)    // 初始化：推挽输出
{
    GPIO_InitTypeDef GPIO_InitStructure;
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);
    GPIO_InitStructure.GPIO_Pin   = GPIO_Pin_5;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Mode  = GPIO_Mode_Out_PP;
    GPIO_Init(GPIOB, &GPIO_InitStructure);
}
// main中: GPIO_SetBits(GPIOB, GPIO_Pin_5); // 亮
//          GPIO_ResetBits(GPIOB, GPIO_Pin_5); // 灭
```

按键检测模板

```
void KEY_Init(void)    // 初始化：上拉输入
{
    GPIO_InitTypeDef GPIO_InitStructure;
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);
    GPIO_InitStructure.GPIO_Pin   = GPIO_Pin_0;
    GPIO_InitStructure.GPIO_Mode  = GPIO_Mode_IPU; // 上拉输入
    GPIO_Init(GPIOA, &GPIO_InitStructure);
}
// 读取: if(GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_0) == 0) // 按下
```

四、GPIO 模式速记

模式常量	用途	考试场景
GPIO_Mode_Out_PP	推挽输出	LED、蜂鸣器
GPIO_Mode_Out_OD	开漏输出	I2C（不考）
GPIO_Mode_AF_PP	复用推挽	USART TX、PWM 输出
GPIO_Mode_AF_OD	复用开漏	I2C（不考）
GPIO_Mode_IN_FLOATING	浮空输入	外部信号输入
GPIO_Mode_IPU	上拉输入	按键（默认高电平）
GPIO_Mode_IPD	下拉输入	按键（默认低电平）

五、必背函数表

函数原型	作用	参数说明
void GPIO_Init(GPIO_TypeDef* GPIOx, GPIO_InitTypeDef* p)	初始化 GPIO	GPIOx: GPIOA~G p: & 结构体变量
void GPIO_SetBits(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)	引脚置高	GPIOx: 端口号 GPIO_Pin: Pin_0~Pin_15
void GPIO_ResetBits(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)	引脚置低	同上
uint8_t GPIO_ReadInputDataBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)	读取输入	返回 0 或 1
void GPIO_WriteBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin, BitAction BitVal)	写单个引脚	BitVal: Bit_SET/Bit_RESET

六、易错点

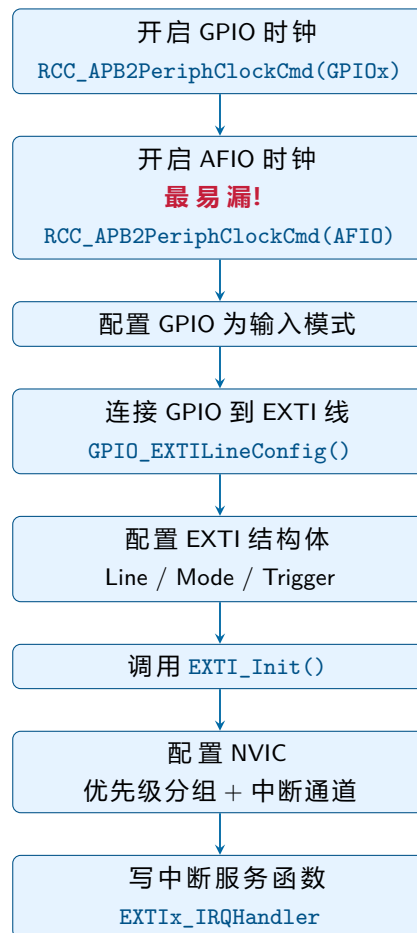
- **GPIO 挂在 APB2 总线上**，用 RCC_APB2PeriphClockCmd
- 输出模式必须写 GPIO_Speed；输入模式可省略
- LED 用 **推挽输出**，不是复用推挽
- 按键用 **上拉输入**（默认高，按下为低）
- 函数名大小写必须一致，考试拼错扣分

第二章 EXTI（外部中断）

一、考试考什么

- 按键触发外部中断
- EXTI 完整配置链
- 中断服务函数书写
- AFIO 时钟（最易漏）
- 中断优先级分组

二、EXTI 配置流程图



三、万能代码框架

```
void EXTI_Config(void)
{
    GPIO_InitTypeDef    GPIO_InitStructure;
    EXTI_InitTypeDef     EXTI_InitStructure;
    NVIC_InitTypeDef     NVIC_InitStructure;

    // 1. 开时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_AFIO, ENABLE); // 别漏AFIO!

    // 2. GPIO输入
    GPIO_InitStructure.GPIO_Pin  = GPIO_Pin_0;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU; // 上拉输入
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    // 3. 连接GPIO到EXTI
    GPIO_EXTILineConfig(GPIO_PortSourceGPIOA, GPIO_PinSource0);

    // 4. EXTI配置
    EXTI_InitStructure.EXTI_Line      = EXTI_Line0;
    EXTI_InitStructure.EXTI_Mode      = EXTI_Mode_Interrupt; // 中断模式
    EXTI_InitStructure.EXTI_Trigger   = EXTI_Trigger_Falling; // 下降沿
    EXTI_InitStructure.EXTI_LineCmd   = ENABLE;
    EXTI_Init(&EXTI_InitStructure);

    // 5. NVIC
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); // 2位抢占 2位响应
    NVIC_InitStructure.NVIC_IRQChannel = EXTI0_IRQn;
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
    NVIC_InitStructure.NVIC_IRQChannelSubPriority       = 0;
    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
    NVIC_Init(&NVIC_InitStructure);
}

// 6. 中断服务函数
void EXTI0_IRQHandler(void)
{
    if(EXTI_GetITStatus(EXTI_Line0) != RESET) // 检查中断源
    {
        // 用户处理代码
        EXTI_ClearITPendingBit(EXTI_Line0); // 清标志位!
    }
}
```

四、配置口诀

EXTI 配置链（必背）：
GPIO → AFIO → EXTI → NVIC
开时钟 → 配 GPIO → 连线 → 配 EXTI → 配 NVIC → 写 ISR

五、必背函数表

函数原型	作用	参数说明
void GPIO_EXTILineConfig(uint8_t GPIO_PortSource, uint8_t GPIO_PinSource)	GPIO→EXTI 映射	PortSource: GPIO_PortSourceGPIOx PinSource: GPIO_PinSource0~15
void EXTI_Init(EXTI_InitTypeDef* p)	初始化 EXTI	p: & 结构体变量
ITStatus EXTI_GetITStatus(uint32_t EXTI_Line)	检查中断标志	EXTI_Line: EXTI_Line0~Line15 返回 SET / RESET
void EXTI_ClearITPendingBit(uint32_t EXTI_Line)	清除中断标志	同上
void NVIC_PriorityGroupConfig(uint32_t NVIC_PriorityGroup)	优先级分组	Group_0~Group_4
void NVIC_Init(NVIC_InitTypeDef* p)	初始化 NVIC	p: & 结构体变量

六、中断通道名对照表

Pin	EXTI 线	IRQn	ISR 函数名
PA0/PB0/PC0	EXTI_Line0	EXTI0_IRQn	EXTI0_IRQHandler
PA1/PB1/PC1	EXTI_Line1	EXTI1_IRQn	EXTI1_IRQHandler
PA2/PB2/PC2	EXTI_Line2	EXTI2_IRQn	EXTI2_IRQHandler
PA3/PB3/PC3	EXTI_Line3	EXTI3_IRQn	EXTI3_IRQHandler
PA4/PB4/PC4	EXTI_Line4	EXTI4_IRQn	EXTI4_IRQHandler
5-9	各自 Line	EXTI9_5_IRQn	EXTI9_5_IRQHandler
10-15	各自 Line	EXTI15_10_IRQn	EXTI15_10_IRQHandler

注意：EXTI5~9 共用一个中断向量！EXTI10~15 共用一个！
考试时 ISR 里必须分别判断各自 Line 的 ITStatus！

七、易错点

- 忘记开 **AFIO 时钟**——最高频扣分点
- 忘记 `GPIO_EXTILineConfig()` (GPIO → EXTI 映射)
- 中断服务函数名写错 (对照上表!)
- 忘记在 ISR 里清中断标志位
- 优先级分组写错
- EXTI5-9 / EXTI10-15 共用一个 ISR, 需要分别判断 Line

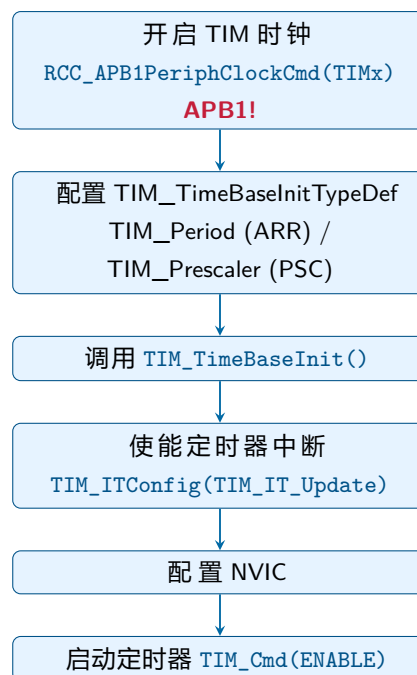
第三章 TIM（定时器）

一、考试考什么

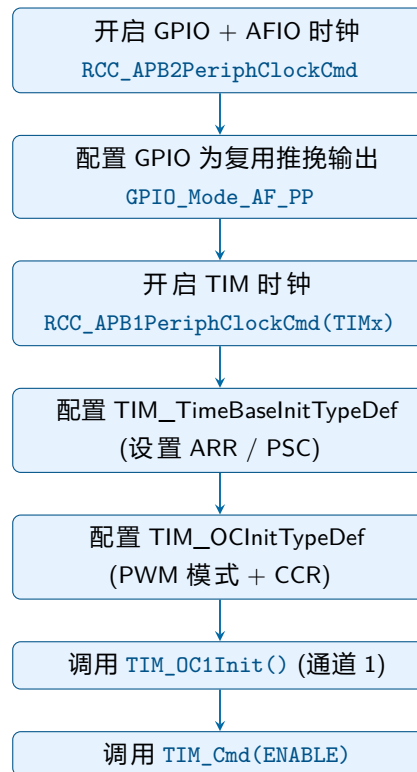
- 定时器中断（1ms / 1s 定时）
- PWM 输出（方波占空比可调）
- ARR 和 PSC 计算
- CCR 与占空比关系
- 定时器初始化结构体

注意：本章不包含输入捕获！仅考察定时器中断 + PWM 输出。

二、定时器中断配置流程图



三、PWM 配置流程图



四、ARR & PSC 作用（考试必背）

参数	名称	调什么	计算公式
<code>TIM_Prescaler</code>	PSC（预分频器）	调频率	$f_{CK_CNT} = f_{CLK}PSC + 1$
<code>TIM_Period</code>	ARR（自动重载）	调周期	$T = (ARR + 1) \times (PSC + 1)f_{CLK}$
<code>TIM_Pulse</code>	CCR（比较值）	调占空比	$Duty = \frac{CCR}{ARR + 1} \times 100\%$

考试秒记：

想改频率 → 改 **PSC**

想改占空比 → 改 **CCR**

想改周期 → 改 **ARR**

默认 $f_{CLK} = 72MHz$ (STM32F103)

五、万能代码框架

定时器中断（1ms）

```
void TIM2_Init(void)    // TIM2在 APB1 上
{
    TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;
    NVIC_InitTypeDef        NVIC_InitStructure;

    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE); // APB1!

    TIM_TimeBaseStructure.TIM_Period          = 999;    // ARR
    TIM_TimeBaseStructure.TIM_Prescaler       = 71;     // PSC
    // 72MHz/(71+1)/(999+1) = 1kHz = 1ms
    TIM_TimeBaseStructure.TIM_ClockDivision   = TIM_CKD_DIV1;
    TIM_TimeBaseStructure.TIM_CounterMode     = TIM_CounterMode_Up;
    TIM_TimeBaseInit(TIM2, &TIM_TimeBaseStructure);

    TIM_ITConfig(TIM2, TIM_IT_Update, ENABLE); // 使能更新中断

    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
    NVIC_InitStructure.NVIC_IRQChannel = TIM2_IRQn;
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
    NVIC_InitStructure.NVIC_IRQChannelSubPriority       = 0;
    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
    NVIC_Init(&NVIC_InitStructure);

    TIM_Cmd(TIM2, ENABLE); // 启动定时器!
}

void TIM2_IRQHandler(void)
{
    if(TIM_GetITStatus(TIM2, TIM_IT_Update) != RESET)
    {
        // 用户代码
        TIM_ClearITPendingBit(TIM2, TIM_IT_Update); // 清标志!
    }
}
```

PWM 输出 (TIM3 CH1, PA6)

```
void PWM_Init(void)
{
    GPIO_InitTypeDef  GPIO_InitStructure;
    TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;
    TIM_OCInitTypeDef  TIM_OCInitStructure;

    // 1. GPIO + AFIO时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA|RCC_APB2Periph_AFIO, ENABLE);
    // 2. GPIO复用推挽
    GPIO_InitStructure.GPIO_Pin   = GPIO_Pin_6;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Mode  = GPIO_Mode_AF_PP; // 复用推挽!
    GPIO_Init(GPIOA, &GPIO_InitStructure);
    // 3. TIM时钟
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM3, ENABLE); // APB1!
    // 4. 时基
    TIM_TimeBaseStructure.TIM_Period      = 999; // ARR
    TIM_TimeBaseStructure.TIM_Prescaler   = 71; // PSC (72MHz/72=1MHz)
    TIM_TimeBaseStructure.TIM_ClockDivision = TIM_CKD_DIV1;
    TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_TimeBaseInit(TIM3, &TIM_TimeBaseStructure);
    // 5. PWM模式
    TIM_OCInitStructure.TIM_OCMode      = TIM_OCMode_PWM1;
    TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;
    TIM_OCInitStructure.TIM_Pulse       = 500; // CCR = 50%占空比
    TIM_OCInitStructure.TIM_OCPolarity  = TIM_OCPolarity_High;
    TIM_OC1Init(TIM3, &TIM_OCInitStructure); // CH1
    // 6. 启动
    TIM_Cmd(TIM3, ENABLE);

    // 运行时改占空比:
    // TIM_SetCompare1(TIM3, 800); // 80%占空比
}
```

六、必背函数表

函数原型	作用	参数说明
void TIM_TimeBaseInit(TIM_TypeDef* TIMx, TIM_TimeBaseInitTypeDef* p)	初始化时基 (ARR/PSC)	TIMx: TIM1~TIM4 p: & 结构体变量
void TIM_OC1Init(TIM_TypeDef* TIMx, TIM_OCInitTypeDef* p)	初始化 PWM CH1	TIMx: 定时器号 p: & 结构体变量
void TIM_OC2Init((同上)	初始化 PWM CH2	同 OC1Init
void TIM_OC3Init((同上)	初始化 PWM CH3	同 OC1Init
void TIM_OC4Init((同上)	初始化 PWM CH4	同 OC1Init
void TIM_ITConfig(TIM_TypeDef* TIMx, uint16_t TIM_IT, FunctionalState NewState)	使能/关闭中 断	TIM_IT: TIM_IT_Update(更 新中断) NewState: EN- ABLE/DISABLE
void TIM_Cmd(TIM_TypeDef* TIMx, FunctionalState NewState)	启动/停止定 时器	NewState: EN- ABLE/DISABLE
ITStatus TIM_GetITStatus(TIM_TypeDef* TIMx, uint16_t TIM_IT)	检查中断标志	返回 SET / RESET
void TIM_ClearITPendingBit(TIM_TypeDef* TIMx, uint16_t TIM_IT)	清除中断标志	同上
void TIM_SetCompare1(TIM_TypeDef* TIMx, uint16_t Compare1)	修改 CCR1 (调占空比)	Compare1: 0~ARR 值 占 空 比 =CCR/(ARR+1)

七、TIM 对应时钟总线

定时器	时钟总线	
TIM1, TIM8	APB2	RCC_APB2PeriphClockCmd
TIM2, TIM3, TIM4, TIM5	APB1	RCC_APB1PeriphClockCmd

高频错误：TIM2/TIM3/TIM4 挂在 APB1，不是 APB2！

考试中写 RCC_APB2PeriphClockCmd(RCC_APB2Periph_TIM2) 直接扣分！

八、PWM 通道与引脚映射（考试常用）

定时器	通道	默认引脚	重映射
TIM1 CH1	OC1	PA8	—
TIM2 CH1	OC1	PA0	PA15
TIM3 CH1	OC1	PA6	PB4
TIM3 CH2	OC2	PA7	PB5
TIM4 CH1	OC1	PB6	—

九、易错点

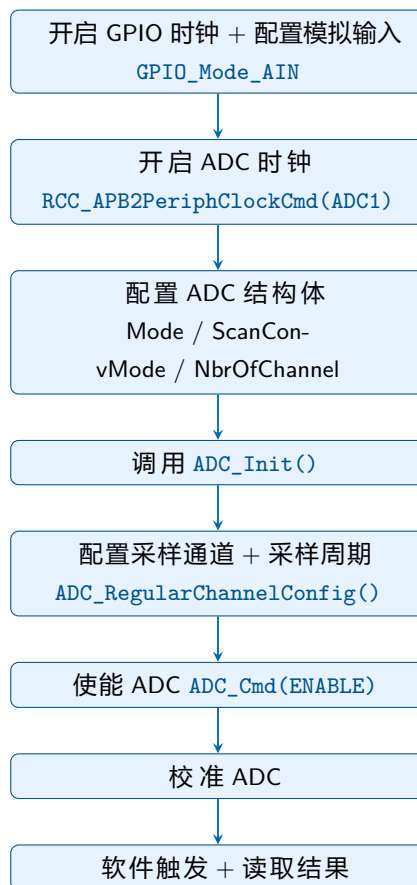
- TIM2/3/4 在 APB1，TIM1/8 在 APB2——写错总线
- PWM 忘记设 GPIO 为 AF_PP（复用推挽）
- 忘记 TIM_Cmd(ENABLE)——定时器不跑
- 忘记 TIM_ITConfig 使能中断
- 忘记在 ISR 里清中断标志位
- PWM 忘记配置 TIM_OCInitTypeDef
- PSC 和 ARR 的值写反（PSC 调频率，ARR 调周期）

第四章 ADC（模数转换器）

一、考试考什么

- ADC 单通道软件触发采样
- ADC 初始化结构体配置
- 读取 ADC 转换值
- 转换结果计算公式

二、ADC 配置流程图



三、万能代码框架

ADC 初始化 + 读取

```
void ADC1_Init(void)
{
    GPIO_InitTypeDef  GPIO_InitStructure;
    ADC_InitTypeDef    ADC_InitStructure;

    // 1. 时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA|RCC_APB2Periph_ADC1, ENABLE);

    // 2. GPIO --- 模拟输入!
    GPIO_InitStructure.GPIO_Pin  = GPIO_Pin_1;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AIN; // 模拟输入
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    // 3. ADC配置
    ADC_InitStructure.ADC_Mode          = ADC_Mode_Independent; // 独立模式
    ADC_InitStructure.ADC_ScanConvMode= DISABLE;                // 单通道 不扫描
    ADC_InitStructure.ADC_ContinuousConvMode = DISABLE; // 单次转换
    ADC_InitStructure.ADC_ExternalTrigConv = ADC_ExternalTrigConv_None; // 软件触发
    ADC_InitStructure.ADC_DataAlign     = ADC_DataAlign_Right; // 右对齐
    ADC_InitStructure.ADC_NbrOfChannel= 1;                      // 1个通道
    ADC_Init(ADC1, &ADC_InitStructure);

    // 4. 通道配置: 通道1, 采样周期55.5周期, 序列1
    ADC_RegularChannelConfig(ADC1, ADC_Channel_1, 1,
                            ADC_SampleTime_55Cycles5);

    // 5. 使能 + 校准
    ADC_Cmd(ADC1, ENABLE);
    ADC_ResetCalibration(ADC1);
    while(ADC_GetResetCalibrationStatus(ADC1));
    ADC_StartCalibration(ADC1);
    while(ADC_GetCalibrationStatus(ADC1));
}

// 读取函数
uint16_t ADC_Read(void)
{
    ADC_SoftwareStartConvCmd(ADC1, ENABLE); // 软件触发
    while(!ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC)); // 等转换结束
    return ADC_GetConversionValue(ADC1); // 返回结果
}
```

四、常见函数速查表

函数原型	作用	参数说明
void ADC_Init(ADC_TypeDef* ADCx, ADC_InitTypeDef* p)	初始化 ADC	ADCx: ADC1~ADC3 p: & 结构体变量
void ADC-RegularChannelConfig(ADC_TypeDef* ADCx, uint8_t ADC_Channel, uint8_t Rank, uint8_t ADC_SampleTime)	配置规则通道 + 采样周期	ADC_Channel: Channel_0~17 Rank: 1~16 (序号) SampleTime: 55Cycles5 等
void ADC_Cmd(ADC_TypeDef* ADCx, FunctionalState NewState)	使能/关闭 ADC	NewState: ENABLE/DISABLE
void ADC_SoftwareStartConvCmd(ADC_TypeDef* ADCx, FunctionalState NewState)	软件触发转换	NewState: ENABLE
FlagStatus ADC_GetFlagStatus(ADC_TypeDef* ADCx, uint8_t ADC_FLAG)	检查标志位	FLAG: ADC_FLAG_EOC(转换结束) 返回 SET / RESET
uint16_t ADC_GetConversionValue(ADC_TypeDef* ADCx)	读取转换结果	返回 0~4095(12 位)
void ADC_ResetCalibration(ADC_TypeDef* ADCx)	复位校准	—
void ADC_StartCalibration(ADC_TypeDef* ADCx)	开始校准	—
FlagStatus ADC_GetCalibrationStatus(ADC_TypeDef* ADCx)	校准完成?	返回 SET/RESET

五、易错点

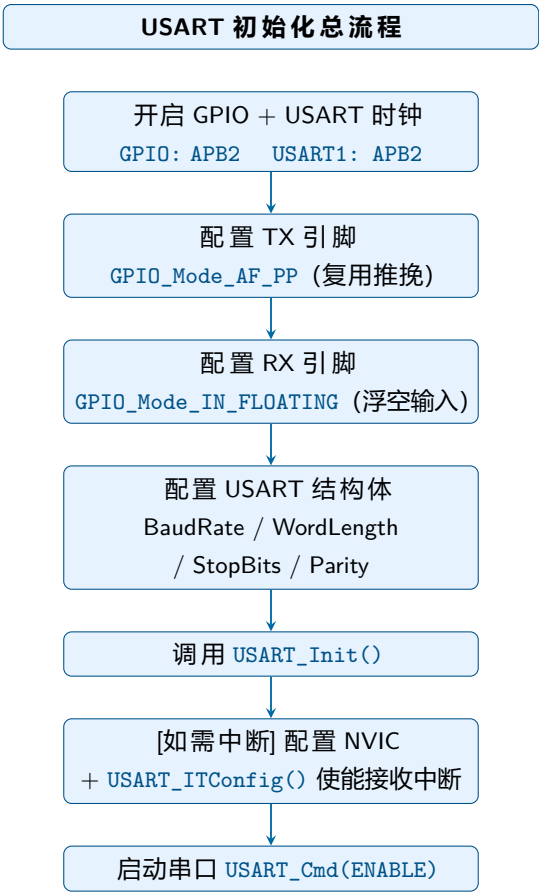
- **ADC GPIO 模式是 AIN (模拟输入)，不是 IN_FLOATING**
- 单通道时 ADC_ScanConvMode = DISABLE
- 软件触发时 ADC_ExternalTrigConv = ADC_ExternalTrigConv_None
- 忘记 ADC 校准流程 (复位校准 → 开始校准 → 等完成)
- 忘记 ADC_Cmd(ENABLE)
- 读取前没有等 EOC 标志位

第五章 USART（串口通信） [全书最详细]

一、考试考什么

- 串口发送一个字节
 - 串口发送字符串
 - 串口接收一个字节（轮询）
- 串口接收中断
 - printf 重定向
 - 中断服务函数书写

二、USART 配置流程图



三、USART 收发流程图

发送流程

```
USART_SendData(USART1, data)
```

```
while(!USART_GetFlagStatus(  
    USART_FLAG_TC)) 等待发送完成
```

接收流程 (轮询)

```
while(!USART_GetFlagStatus(  
    USART_FLAG_RXNE)) 等待接收
```

```
data = USART_ReceiveData(USART1)
```

四、万能代码框架

USART1 初始化（完整版）

```
void USART1_Config(void)
{
    GPIO_InitTypeDef    GPIO_InitStructure;
    USART_InitTypeDef    USART_InitStructure;
    NVIC_InitTypeDef     NVIC_InitStructure;

    // 1. 开时钟 (GPIOA + USART1 都在 APB2)
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA |
                           RCC_APB2Periph_USART1, ENABLE);

    // 2. TX --- PA9, 复用推挽
    GPIO_InitStructure.GPIO_Pin   = GPIO_Pin_9;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Mode  = GPIO_Mode_AF_PP; // TX = AF_PP
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    // 3. RX --- PA10, 浮空输入
    GPIO_InitStructure.GPIO_Pin   = GPIO_Pin_10;
    GPIO_InitStructure.GPIO_Mode  = GPIO_Mode_IN_FLOATING; // RX = IN_FLOATING
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    // 4. USART参数
    USART_InitStructure.USART_BaudRate    = 9600;
    USART_InitStructure.USART_WordLength = USART_WordLength_8b;
    USART_InitStructure.USART_StopBits   = USART_StopBits_1;
    USART_InitStructure.USART_Parity     = USART_Parity_No;
    USART_InitStructure.USART_HardwareFlowControl
        = USART_HardwareFlowControl_None;
    USART_InitStructure.USART_Mode = USART_Mode_Tx | USART_Mode_Rx;
    USART_Init(USART1, &USART_InitStructure);

    // 5. NVIC (中断接收才需要)
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
    NVIC_InitStructure.NVIC_IRQChannel = USART1_IRQn;
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
    NVIC_InitStructure.NVIC_IRQChannelSubPriority       = 0;
    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
    NVIC_Init(&NVIC_InitStructure);

    // 6. 使能接收中断 (中断接收才需要)
    USART_ITConfig(USART1, USART_IT_RXNE, ENABLE);

    // 7. 启动串口!
    USART_Cmd(USART1, ENABLE);
}
```

发送一个字节

```
void USART_SendByte(uint8_t data)
{
    USART_SendData(USART1, data);
    while(USART_GetFlagStatus(USART1, USART_FLAG_TC) == RESET);
    // 等发送完成
}
```

发送字符串

```
void USART_SendString(char *str)
{
    while(*str)
    {
        USART_SendData(USART1, *str++);
        while(USART_GetFlagStatus(USART1, USART_FLAG_TC) == RESET);
    }
}
```

轮询接收一个字节

```
uint8_t USART_ReceiveByte(void)
{
    while(USART_GetFlagStatus(USART1, USART_FLAG_RXNE) == RESET);
    // 等 RXNE=1 (接收非空)
    return (uint8_t)USART_ReceiveData(USART1);
}
```

中断接收 + ISR

```
uint8_t rx_data; // 全局变量接收数据

void USART1_IRQHandler(void)
{
    if(USART_GetITStatus(USART1, USART_IT_RXNE) != RESET)
    {
        rx_data = USART_ReceiveData(USART1);
        // 读取DR会自动清除RXNE标志
    }
}
```

printf 重定向

```
// 需 #include <stdio.h>
int fputc(int ch, FILE *f)
{
    USART_SendData(USART1, (uint8_t)ch);
    while(USART_GetFlagStatus(USART1, USART_FLAG_TC) == RESET);
    return ch;
}
// 之后可直接用 printf("Hello\r\n");
```

五、配置口诀

USART 初始化口诀：

开时钟 → 配 TX(AF_PP) → 配 RX(IN_FLOATING) → 配波特率 → 配 NVIC → 开中断 → USART_Cmd

USART 收发口诀：

发：SendData → 等 TC

收：等 RXNE → ReceiveData

printf 重定向：

写 fputc → 包含 stdio.h → 直接 printf

六、必背函数表

函数原型	作用	参数说明
void USART_Init(USART_TypeDef* USARTx, USART_InitTypeDef* p)	初始化 USART	USARTx: USART1~USART3 p: & 结构体变量
void USART_Cmd(USART_TypeDef* USARTx, FunctionalState NewState)	使能/关闭 USART	必须写 ENABLE!
void USART_SendData(USART_TypeDef* USARTx, uint16_t Data)	发送一字节	Data: 8 位数据 (0~255)
uint16_t USART_ReceiveData(USART_TypeDef* USARTx)	读取一字节	返回接收到的 8 位数据
FlagStatus USART_GetFlagStatus(USART_TypeDef* USARTx, uint16_t USART_FLAG)	查询标志位	FLAG: TC(发送完成) RXNE(接收非空) TXE(发送空)
void USART_ITConfig(USART_TypeDef* USARTx, uint16_t USART_IT, FunctionalState NewState)	使能/关闭中断源	USART_IT: RXNE(接收中断) NewState: ENABLE/DISABLE
ITStatus USART_GetITStatus(USART_TypeDef* USARTx, uint16_t USART_IT)	检查中断状态	在 ISR 中使用, 返回 SET/RESET
void USART_ClearITPendingBit(USART_TypeDef* USARTx, uint16_t USART_IT)	清除中断挂起	读 DR 后 RXNE 自动清除

七、USART 标志位速查

标志位	含义	使用场景
USART_FLAG_TXE	发送数据寄存器空	可继续发送下一字节
USART_FLAG_TC	发送完成	等当前字节完全发出
USART_FLAG_RXNE	接收数据寄存器非空	有数据到达, 可读取
USART_IT_RXNE	接收中断使能	中断方式接收
USART_IT_TXE	发送中断使能	中断方式发送

考试注意区分: **TXE** (空, 可继续发) vs **TC** (完成, 已发出)
发送字节后一般等 TC; 连续发送时用 TXE 更高效 (但考试多用 TC)。

八、USART 常用引脚

串口	TX	RX	时钟总线
USART1	PA9	PA10	APB2
USART2	PA2	PA3	APB1
USART3	PB10	PB11	APB1

九、易错点

- 忘记 `USART_Cmd(ENABLE)`——最高频错误
- **TX 用 `AF_PP`, RX 用 `IN_FLOATING`**——模式写反
- **USART1 在 APB2; USART2/3 在 APB1**
- 忘记 `USART_ITConfig()` 使能中断
- 发送后没等 TC/TXE 标志位
- 接收前没等 RXNE 标志位
- `printf` 重定向时忘记 `#include <stdio.h>`
- 中断函数名写错 (`USART1_IRQHandler`)
- 串口模式忘了写 `USART_Mode_Tx` | `USART_Mode_Rx`

附录 A 考试万能配置总口诀

五大模块配置链（考试必背）

GPIO: 开时钟 → 配 Pin/Speed/Mode → GPIO_Init	开时钟 → GPIO(AF_PP) → TIM 时基 → TIM_OC → TIM_Cmd
EXTI: 开时钟 → GPIO → AFIO → EXTI → NVIC → ISR	ADC: 开时钟 → GPIO(AIN) → ADC_Init → 通道配置 → 校准 → 触发
TIM 中断: 开时钟 → 配 ARR/PSC → TIM_ITConfig → NVIC → TIM_Cmd	USART: 开时钟 → TX(AF_PP) → RX(IN_FLOATING) → 波特率 → NVIC → 开中断 → USART_Cmd
PWM:	

附录 B NVIC 配置模板

```
NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);  
// 常用: Group_2 = 2位抢占 + 2位响应  
  
NVIC_InitStructure.NVIC_IRQChannel = XXXX_IRQn;  
NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = x; // 0~3  
NVIC_InitStructure.NVIC_IRQChannelSubPriority = y; // 0~3  
NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;  
NVIC_Init(&NVIC_InitStructure);
```

附录 C 中断服务函数名速查

外设	ISR 函数名
EXTI0	EXTI0_IRQHandler
EXTI1	EXTI1_IRQHandler
EXTI2	EXTI2_IRQHandler
EXTI3	EXTI3_IRQHandler
EXTI4	EXTI4_IRQHandler
EXTI5-9	EXTI9_5_IRQHandler
EXTI10-15	EXTI15_10_IRQHandler
TIM2	TIM2_IRQHandler
TIM3	TIM3_IRQHandler
TIM4	TIM4_IRQHandler
USART1	USART1_IRQHandler
USART2	USART2_IRQHandler

附录 D 时钟总线速查

RCC 时钟使能函数（考试必背）

函数原型	总线	常用外设
void RCC_APB2PeriphClockCmd(uint32_t RCC_APB2Periph, FunctionalState NewState)	APB2 72MHz	GPIOA~G, AFIO, USART1, ADC1~3, TIM1, TIM8, SPI1
void RCC_APB1PeriphClockCmd(uint32_t RCC_APB1Periph, FunctionalState NewState)	APB1 36MHz	TIM2~TIM7, USART2~USART3, SPI2~SPI3, I2C

常用外设时钟参数

外设	时钟参数	总线
GPIOA~G	RCC_APB2Periph_GPIOx	APB2
AFIO	RCC_APB2Periph_AFIO	APB2
USART1	RCC_APB2Periph_USART1	APB2
ADC1	RCC_APB2Periph_ADC1	APB2
TIM1	RCC_APB2Periph_TIM1	APB2
TIM2	RCC_APB1Periph_TIM2	APB1
TIM3	RCC_APB1Periph_TIM3	APB1
TIM4	RCC_APB1Periph_TIM4	APB1
USART2	RCC_APB1Periph_USART2	APB1

APB1 和 APB2 搞反，考试直接扣分！
口诀：GPIO、USART1、ADC ——APB2
TIM2/3/4、USART2/3 ——APB1

附录 E 考试高频扣分点 TOP 10

- #1 忘记开外设时钟（GPIO / USART / TIM / ADC 时钟无一例外都要开）
- #2 忘记开 AFIO 时钟（EXTI 章节专属）
- #3 TIM2/3/4 写在 APB2（应该用 RCC_APB1PeriphClockCmd）
- #4 TX 和 RX 的 GPIO 模式写反（TX = AF_PP，RX = IN_FLOATING）
- #5 PWM 的 GPIO 模式写成 Out_PP 而不是 AF_PP
- #6 忘记 XXX_Cmd(ENABLE)（USART_Cmd、TIM_Cmd、ADC_Cmd 都必须写）
- #7 ISR 函数名写错（对照附录 C）
- #8 忘记在 ISR 里清中断标志位
- #9 ADC GPIO 模式写成 IN_FLOATING 而不是 AIN
- #10 结构体成员名拼写错误（考试手写代码不能靠 IDE 补全）

— 祝考试顺利 —

看到题目 → 回忆配置链 → 套模板 → 不漏步骤

考前对照附录 E 逐条自查，确保没有遗漏